FitPal — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── src/
│   ├── memory/
│   │   ├── __init__.py
│   │   ├── store.py
│   │   └── writer.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   ├── chat.txt
│   │   └── writer.txt
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── graph.py
│   ├── nodes.py
│   └── state.py
├── tests/
│   ├── integration/
│   │   └── test_live_memory.py
│   └── unit/
│       ├── test_config.py
│       └── test_store.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
└── requirements.txt
```

Setup files

`requirements.txt`: `langgraph`, `langgraph-checkpoint-sqlite`, `langchain-openai`, `python-dotenv`, `pydantic`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `*.sqlite`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, prompts

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
    log_level: str


def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value


def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os


def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py
PROMPTS_DIR = "src/prompts"


def load_prompt(name: str) -> str:
    with open(PROMPTS_DIR + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

MODEL_NAME = "gpt-4o-mini"


def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model=MODEL_NAME, temperature=0, api_key=config.openai_api_key
    )
```

The state

```
# src/state.py
from typing import Annotated, TypedDict

from langgraph.graph.message import add_messages


class ChatState(TypedDict):
    messages: Annotated[list, add_messages]
    user_id: str
    recalled: str
```

The two memory systems

```
# src/memory/store.py
# Long-term memory: durable facts about a user (goal, equipment, injuries),
# independent of any conversation. Keyed by user_id. Same key = update.
from langgraph.store.memory import InMemoryStore


def build_store():
    return InMemoryStore()


def namespace_for(user_id: str) -> tuple:
    return ("fitness", user_id)


def recall_facts(store, user_id: str) -> list:
    facts = []
    for item in store.search(namespace_for(user_id)):
        facts.append(item.value["fact"])
    return facts


def save_fact(store, user_id: str, key: str, fact: str) -> None:
    store.put(namespace_for(user_id), key, {"fact": fact})


def forget_user(store, user_id: str) -> int:
    namespace = namespace_for(user_id)
    deleted = 0
    for item in store.search(namespace):
        store.delete(namespace, item.key)
```

```
        deleted = deleted + 1
    return deleted
```

```
# src/memory/writer.py
# Decide whether the user's last message holds a durable fitness fact worth
# keeping, and under which stable key. The model returns a checked decision.
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel

from src.utils.prompts import load_prompt


class MemoryDecision(BaseModel):
    should_remember: bool
    key: str
    fact: str


def decide_memory(model, message: str) -> MemoryDecision:
    prompt = ChatPromptTemplate.from_template(load_prompt("writer"))
    chain = prompt | model.with_structured_output(MemoryDecision)
    return chain.invoke({"message": message})
```

```
# src/prompts/writer.txt
Decide whether the user's message contains a DURABLE fitness fact worth
remembering long-term about this user.

Remember: a training goal, available equipment, an injury or limitation, a
standing schedule preference.
Do NOT remember: how they felt today, one-off questions, small talk, a
single workout they just did.

If worth remembering, set should_remember true, choose a short STABLE key
(use "goal" for their training goal, "equipment", "injury", "schedule" -
reuse the same key so a change updates it), and write the fact as a short
sentence ("Goal is a 10k", "Injury: bad left knee, avoid deep squats").
Otherwise set should_remember false with empty key and fact.

User message: {message}
```

The nodes

```
# src/nodes.py
# recall_node reads the Store (injected via the store parameter). chat_node
# coaches with recalled facts in its system prompt. write_back_node decides
# what new durable fact to save. The model is bound by closures; the store
# is injected because recall/write_back declare a store parameter.
from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.store.base import BaseStore

from src.memory.store import recall_facts, save_fact
from src.memory.writer import decide_memory
from src.utils.prompts import load_prompt


def recall_node(state, *, store: BaseStore):
    facts = recall_facts(store, state["user_id"])
    return {"recalled": "; ".join(facts)}


def make_chat_node(model):
    def chat_node(state):
        recalled = state.get("recalled") or "nothing yet"
        system = load_prompt("chat").replace("{recalled}", recalled)
        messages = [SystemMessage(content=system)] + state["messages"]
        reply = model.invoke(messages)
        return {"messages": [reply]}
    return chat_node


def make_write_back_node(model):
    def write_back_node(state, *, store: BaseStore):
        last_user = None
        for message in reversed(state["messages"]):
            if isinstance(message, HumanMessage):
                last_user = message
                break
        if last_user is None:
            return {}
        decision = decide_memory(model, last_user.content)
        if decision.should_remember:
            save_fact(store, state["user_id"], decision.key, decision.fact)
        return {}
    return write_back_node
```

```
# src/prompts/chat.txt
You are FitPal, a friendly, careful fitness coach. Give safe, practical
advice. ALWAYS respect the user's known injuries and equipment.
```

```
What you remember about this user (may be empty):
{recalled}
```

The graph

```
# src/graph.py
from langgraph.graph import END, START, StateGraph

from src.nodes import make_chat_node, make_write_back_node, recall_node
from src.state import ChatState


def build_graph(model, store, checkpointer):
    graph = StateGraph(ChatState)
    graph.add_node("recall", recall_node)
    graph.add_node("chat", make_chat_node(model))
    graph.add_node("write_back", make_write_back_node(model))
    graph.add_edge(START, "recall")
    graph.add_edge("recall", "chat")
    graph.add_edge("chat", "write_back")
    graph.add_edge("write_back", END)
    return graph.compile(checkpointer=checkpointer, store=store)
```

main.py

```
# main.py
import sys

from langchain_core.messages import HumanMessage
from langgraph.checkpoint.sqlite import SqliteSaver

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.memory.store import build_store, forget_user
from src.models.llm import get_model

USER = "me"
THREAD_ID = "session-1"
DB_PATH = "fitpal.sqlite"
```

```
def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)
    model = get_model(config)
    store = build_store()
    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        app = build_graph(model, store, checkpointer)
        print("FitPal — tell me your goals, or /quit. Type forget_me to wipe.")
        run_config = {"configurable": {"thread_id": THREAD_ID}}
        while True:
            text = input("You: ").strip()
            if text == "/quit":
                break
            if text == "":
                continue
            if text == "forget_me":
                count = forget_user(store, USER)
                print(f"FitPal: forgotten — removed {count} saved facts.")
                continue
            result = app.invoke(
                {"messages": [HumanMessage(content=text)],
                 "user_id": USER, "recalled": ""},
                run_config,
            )
            print("FitPal:", result["messages"][-1].content)


if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "FITPAL_TEST_SETTING"
```

```
def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_empty_setting() -> None:
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")


def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")


if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_store.py
# The Store helpers, no model. Run from the repo root:
#   python -m tests.unit.test_store
from src.memory.store import (
    build_store,
    forget_user,
    recall_facts,
    save_fact,
)


def test_save_then_recall() -> None:
```

```
    store = build_store()
    save_fact(store, "me", "goal", "Goal is a 10k")
    result = "OK"
    if recall_facts(store, "me") != ["Goal is a 10k"]:
        result = "WRONG"
    print(f"case 1 (save then recall)       -> {result}")


def test_recall_is_per_user() -> None:
    store = build_store()
    save_fact(store, "me", "goal", "Goal is a 10k")
    result = "OK"
    if recall_facts(store, "someone_else") != []:
        result = "WRONG"
    print(f"case 2 (another user sees none)  -> {result}")


def test_same_key_updates() -> None:
    store = build_store()
    save_fact(store, "me", "goal", "Goal is a 10k")
    save_fact(store, "me", "goal", "Goal is a half-marathon")
    result = "OK"
    if recall_facts(store, "me") != ["Goal is a half-marathon"]:
        result = "WRONG"
    print(f"case 3 (same key updates)        -> {result}")


def test_forget_removes_all() -> None:
    store = build_store()
    save_fact(store, "me", "goal", "Goal is a 10k")
    save_fact(store, "me", "injury", "Bad left knee")
    removed = forget_user(store, "me")
    result = "OK"
    if removed != 2 or recall_facts(store, "me") != []:
        result = "WRONG"
    print(f"case 4 (forget removes all)      -> {result}")


if __name__ == "__main__":
    test_save_then_recall()
    test_recall_is_per_user()
    test_same_key_updates()
    test_forget_removes_all()
```

```
# tests/integration/test_live_memory.py
# The whole memory story, live. Run from the repo root:
```

```
#   python -m tests.integration.test_live_memory
from langchain_core.messages import HumanMessage
from langgraph.checkpoint.memory import MemorySaver

from src.config import load_config
from src.graph import build_graph
from src.memory.store import build_store, forget_user, recall_facts
from src.models.llm import get_model

USER = "me"


def fresh_app():
    config = load_config()
    model = get_model(config)
    store = build_store()
    return build_graph(model, store, MemorySaver()), store


def say(app, text, thread):
    out = app.invoke(
        {"messages": [HumanMessage(content=text)], "user_id": USER,
         "recalled": ""},
        {"configurable": {"thread_id": thread}},
    )
    return out["messages"][-1].content


def test_recalls_injury_in_new_session() -> None:
    app, store = fresh_app()
    say(app, "My left knee is bad - avoid deep squats. I only have dumbbells.",
        "s1")
    answer = say(app, "Give me a quick leg workout.", "brand-new-session")
    result = "OK"
    if "squat" in answer.lower() and "avoid" not in answer.lower():
        result = "WRONG"
    print(f"case 1 (recalls injury, new session) -> {result}")


def test_chatter_not_saved() -> None:
    app, store = fresh_app()
    say(app, "I felt a bit tired today.", "s1")
    result = "OK"
    if recall_facts(store, USER) != []:
        result = "WRONG"
    print(f"case 2 (chatter not remembered)      -> {result}")
```

```
def test_goal_updates() -> None:
    app, store = fresh_app()
    say(app, "My goal is a 10k.", "s1")
    say(app, "Actually my goal is a half-marathon now.", "s2")
    goals = []
    for fact in recall_facts(store, USER):
        if "goal" in fact.lower() or "marathon" in fact.lower() \
                or "10k" in fact.lower():
            goals.append(fact)
    result = "OK"
    if len(goals) != 1 or "marathon" not in goals[0].lower():
        result = "WRONG"
    print(f"case 3 (goal updates, not dup)       -> {result}")


def test_forget_leaves_nothing() -> None:
    app, store = fresh_app()
    say(app, "My goal is a 10k and I have dumbbells.", "s1")
    forget_user(store, USER)
    result = "OK"
    if recall_facts(store, USER) != []:
        result = "WRONG"
    print(f"case 4 (forget -> nothing left)      -> {result}")


if __name__ == "__main__":
    test_recalls_injury_in_new_session()
    test_chatter_not_saved()
    test_goal_updates()
    test_forget_leaves_nothing()
```

Expected unit output (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (save then recall)       -> OK
case 2 (another user sees none)  -> OK
case 3 (same key updates)        -> OK
case 4 (forget removes all)      -> OK
```

Expected integration output (real model):

```
case 1 (recalls injury, new session) -> OK
case 2 (chatter not remembered)      -> OK
case 3 (goal updates, not dup)       -> OK
case 4 (forget -> nothing left)      -> OK
```

That's the whole project. If a brand-new session already knows your bad knee and your dumbbells, ignores "felt tired today", updates your goal instead of duplicating it, and truly forgets you on `forget_me` — you've built it.

> **A note on persistence:** the checkpointer is `SqliteSaver`, so a conversation survives a restart; the Store is `InMemoryStore`, so user facts live for the life of the process. For user memory that also survives a restart, swap `InMemoryStore` for a file- or database-backed store with the same `put` / `get` / `search` / `delete` interface — no other code changes.